BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC KIẾN TRÚC ĐÀ NẴNG

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN MÔN HỌC

ĐỀ TÀI:

XÂY DỰNG HỆ THỐNG THÔNG TIN CHỦ ĐỘNG CHO HÀNH ĐỘNG CÁ NHÂN TÍCH HỢP TRÍ TUỆ NHÂN TẠO VÀ NHẮC NHỞ GIỌNG NÓI

Sinh viên thực hiện	:	Võ Trọng Tài – MSV: 2351220084 Nguyễn Cao Hậu
Giảng viên hướng dẫn	:	TS. Đỗ Phúc Hảo
Chuyên ngành	:	Công nghệ thông tin
Công nghệ cốt lõi	:	Flutter, Node.js, MongoDB, AI & Text-to-Speech
Năm học	:	2025 – 2026

Đà Nẵng – 2026

LỜI CAM ĐOAN

Chúng tôi xin cam đoan đồ án với đề tài **"Xây dựng hệ thống thông tin chủ động cho hành động cá nhân tích hợp Trí tuệ nhân tạo và Nhắc nhở giọng nói"** là công trình nghiên cứu và phát triển do chính nhóm chúng tôi thực hiện dưới sự hướng dẫn chuyên môn trực tiếp của thầy TS. Đỗ Phúc Hảo.

Các nội dung nghiên cứu lý thuyết, sơ đồ phân tích thiết kế hệ thống, mô hình cơ sở dữ liệu NoSQL MongoDB, toàn bộ mã nguồn ứng dụng đa nền tảng viết bằng Flutter/Node.js, dịch vụ phát giọng nói Text-to-Speech và các kết quả đo đạc hiệu năng trình bày trong báo cáo này là hoàn toàn trung thực, không sao chép từ bất kỳ công trình nào khác chưa được công bố.

Mọi tài liệu tham khảo, thư viện mã nguồn mở và tri thức kế thừa từ các tác giả đi trước đều đã được nhóm trích dẫn, ghi nhận nguồn gốc rõ ràng tại phần Tài liệu tham khảo. Chúng tôi xin chịu hoàn toàn trách nhiệm trước Nhà trường và Khoa Công nghệ Thông tin nếu có bất kỳ sự vi phạm nào về chuẩn mực đạo đức khoa học và bản quyền tác giả.

Đà Nẵng, ngày ...tháng ...năm 2026

Sinh viên thực hiện

Võ Trọng Tài & **Nguyễn Cao Hậu**

LỜI CẢM ƠN

Để hoàn thành đề tài đồ án môn học này một cách trọn vẹn, nhóm chúng em đã nhận được sự hỗ trợ, hướng dẫn và động viên rất lớn từ quý Thầy/Cô cùng gia đình và bạn bè.

Lời đầu tiên, nhóm chúng em xin được gửi lời cảm ơn chân thành và sâu sắc nhất tới thầy **TS. Đỗ Phúc Hảo**. Trong suốt quá trình thực hiện đề tài, thầy đã luôn dành thời gian quý báu để định hướng phương pháp nghiên cứu, góp ý tỉ mỉ về thiết kế kiến trúc hệ thống, chuẩn hóa quy trình phân tích thiết kế và truyền đạt những kinh nghiệm thực tiễn giúp chúng em vượt qua các vướng mắc về mặt kỹ thuật.

Chúng em cũng xin trân trọng cảm ơn Ban Giám hiệu cùng toàn thể quý Thầy/Cô trong Khoa Công nghệ Thông tin – Trường Đại học Kiến trúc Đà Nẵng đã giảng dạy, trang bị cho chúng em những kiến thức nền tảng vững chắc về lập trình di động, quản trị cơ sở dữ liệu, phân tích thiết kế hệ thống và quy trình phát triển phần mềm trong suốt các năm học qua.

Mặc dù nhóm đã nỗ lực hết mình để hoàn thành báo cáo một cách toàn diện nhất, song do hạn chế về mặt thời gian và kinh nghiệm thực tế, báo cáo khó tránh khỏi những thiếu sót nhất định. Nhóm rất mong nhận được những ý kiến đóng góp quý báu từ quý Thầy/Cô để ứng dụng được hoàn thiện hơn trong tương lai.

Đà Nẵng, ngày …tháng …năm 2026

Sinh viên thực hiện

Võ Trọng Tài & **Nguyễn Cao Hậu**

TÓM TẮT ĐỒ ÁN

Trong bối cảnh xã hội hiện đại, tình trạng quá tải thông tin cá nhân (Information Overload) và hiện tượng trì hoãn công việc đang trở thành rào cản lớn đối với hiệu suất học tập và làm việc của nhiều người. Báo cáo này trình bày chi tiết toàn bộ quy trình từ nghiên cứu lý thuyết, phân tích yêu cầu, thiết kế kiến trúc hệ thống, cài đặt mã nguồn cho đến kiểm thử đánh giá hệ thống thông tin quản lý hành động cá nhân chủ động.

Hệ thống được xây dựng dựa trên kiến trúc Client–Server 3 lớp hiện đại với sự kết hợp giữa Framework **Flutter** (tầng giao diện người dùng đa nền tảng), **Node.js/Express** (tầng xử lý logic nghiệp vụ trung gian) và cơ sở dữ liệu hướng tài liệu NoSQL **MongoDB Atlas** (tầng lưu trữ dữ liệu). Điểm đột phá của hệ thống nằm ở khả năng tự động tính toán và phân loại hành động dựa trên ma trận ưu tiên Eisenhower, kết hợp phương pháp lập kế hoạch khối thời gian (Time-Blocking), ứng dụng Trí tuệ nhân tạo (AI) trích xuất ngữ nghĩa tác vụ và cơ chế nhắc nhở chủ động bằng **giọng nói tiếng Việt (Text-to-Speech)**.

Kết quả thử nghiệm thực tế trên tập dữ liệu hơn 1,000 bản ghi cho thấy ứng dụng đạt tốc độ truy vấn trung bình dưới 120ms, độ trễ kích hoạt âm thanh nhắc nhở dưới 50ms, tốc độ khung hình giao diện ổn định ở mức 60fps và vận hành mượt mà trên cả hai hệ điều hành di động Android và iOS.

Từ khóa: Hệ thống thông tin cá nhân, Lập kế hoạch hành động, Flutter, MongoDB, NoSQL, Time-Blocking, Ma trận Eisenhower, Trí tuệ nhân tạo, Nhắc nhở giọng nói.

Mục lục

LỜI CAM ĐOAN 1

LỜI CẢM ƠN 2

TÓM TẮT ĐỒ ÁN 3

Danh mục hình ảnh 7

Danh mục bảng biểu 8

1 TỔNG QUAN VÀ CƠ SỞ LÝ THUYẾT 9

1.1 Đặt vấn đề và Tính cấp thiết của đề tài 9

1.2 Mục tiêu, Đối tượng và Phạm vi nghiên cứu 10

1.2.1 Mục tiêu nghiên cứu 10

1.2.2 Đối tượng và Phạm vi nghiên cứu 10

1.3 Phương pháp nghiên cứu 11

1.4 Cơ sở lý thuyết về Quản lý Hành động Cá nhân 11

1.4.1 Quản lý thông tin cá nhân (Personal Information Management - PIM) 11

1.4.2 Ma trận ưu tiên Eisenhower và Thuật toán tính điểm động 11

1.4.3 Kỹ thuật phân bổ khối thời gian (Time-Blocking) 13

1.5 Cơ sở công nghệ nền tảng 13

1.5.1 Framework Flutter và Kiến trúc quản lý trạng thái Provider 13

1.5.2 Nền tảng Backend Node.js, Express.js và Bảo mật JWT 13

1.5.3 Cơ sở dữ liệu hướng tài liệu MongoDB (NoSQL) 14

1.5.4 Công nghệ tổng hợp giọng nói Text-to-Speech (TTS) 14

2 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 16

2.1 Khảo sát hiện trạng và Đặc tả yêu cầu hệ thống 16

2.1.1 Khảo sát bài toán thực tế 16

2.1.2 Yêu cầu chức năng (Functional Requirements) 16

2.1.3 Yêu cầu phi chức năng (Non-Functional Requirements) 17

2.2 Thiết kế Kiến trúc Tổng thể Hệ thống 17

2.3 Thiết kế các Sơ đồ Phân tích Hệ thống (UML & DFD) 19

2.3.1 Sơ đồ Ca sử dụng tổng quát (Use Case Diagram) 19

2.3.2 Đặc tả chi tiết các Ca sử dụng chính 20

2.3.3 Sơ đồ Luồng dữ liệu (DFD Cấp 1) và Sơ đồ Tuần tự (Sequence Diagram) 21

2.4 Thiết kế Cơ sở dữ liệu MongoDB (NoSQL Schema) 22

2.5 Thiết kế Giao diện RESTful API 26

3 XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG 27

3.1 Môi trường phát triển và Công cụ triển khai 27

3.2 Tổ chức cấu trúc mã nguồn 27

3.2.1 Cấu trúc mã nguồn tầng Backend (Node.js/Express) 27

3.2.2 Cấu trúc mã nguồn ứng dụng di động (Flutter) 28

3.3 Cài đặt các Mô-đun Nghiệp vụ Cốt lõi 28

3.3.1 Cài đặt Schema và Thuật toán tính điểm ưu tiên tại Backend . . . 28

3.3.2 Cài đặt Dịch vụ Nhắc nhở bằng Giọng nói (Text-to-Speech Service) 29

3.3.3 Cài đặt Bộ lập lịch Thông báo nền kết hợp Kích hoạt Giọng nói . . 30

3.4 Triển khai Giao diện Người dùng Ứng dụng 32

4 TRÍ TUỆ NHÂN TẠO (AI) VÀ TỰ ĐỘNG HÓA 34

4.1 Vai trò của Trí tuệ Nhân tạo trong Hệ thống Thông tin Cá nhân 34

4.2 Kiến trúc Mô-đun Xử lý Ngôn ngữ Tự nhiên (AI NLP Pipeline) 34

4.3 Thuật toán Gợi ý Khung giờ Làm việc Thông minh (Smart Time-Blocking) 35

5 KIỂM THỬ, ĐÁNH GIÁ VÀ KẾT LUẬN 37

5.1 Môi trường và Kịch bản Kiểm thử Hệ thống 37

5.2 Kết quả Kiểm thử Chức năng (Functional Testing) 38

5.3 Đánh giá Hiệu năng và Tốc độ Phản hồi Hệ thống 39

5.4 Kết luận và Hướng phát triển 39

5.4.1 Kết luận chung 39

5.4.2 Hạn chế còn tồn tại 40

5.4.3 Hướng phát triển trong tương lai 40

Tài liệu tham khảo 41

Danh sách hình vẽ

1.1 Mô hình phân loại hành động 4 phần tư theo Ma trận Eisenhower. 12

2.1 Sơ đồ kiến trúc hệ thống 3 lớp tích hợp AI và Dịch vụ giọng nói. 18

2.2 Sơ đồ Ca sử dụng (Use Case Diagram) tổng quát của hệ thống. 19

2.3 Sơ đồ luồng dữ liệu (DFD Cấp 1) của quy trình quản lý và nhắc nhở hành động. 21

2.4 Sơ đồ tuần tự (Sequence Diagram) chức năng tạo hành động và kích hoạt giọng nói. 22

2.5 Sơ đồ thực thể liên kết (ERD) giữa các Collections trong MongoDB. 23

3.1 Giao diện màn hình chính hiển thị danh sách hành động theo Ma trận Eisenhower. 33

4.1 Kiến trúc đường ống xử lý ngôn ngữ tự nhiên (AI NLP Pipeline) trích xuất hành động. 34

Danh sách bảng

1.1 So sánh đặc tính kỹ thuật giữa Cơ sở dữ liệu quan hệ (SQL) và MongoDB (NoSQL). 14

2.1 Bảng đặc tả chi tiết các ca sử dụng trọng tâm của hệ thống. 20

2.2 Đặc tả chi tiết cấu trúc Document của Collection `users`. 24

2.3 Đặc tả chi tiết cấu trúc Document của Collection `actions`. 25

2.4 Danh mục thiết kế các Endpoint RESTful API cốt lõi của hệ thống. 26

4.1 Minh họa kết quả phân tích và bóc tách dữ liệu tự động của mô-đun AI. . 36

5.1 Bảng tổng hợp kết quả kiểm thử các ca chức năng trọng tâm của hệ thống. 38

5.2 Kết quả đo đạc hiệu năng thời gian phản hồi trung bình của hệ thống. . . 39

Chương 1

TỔNG QUAN VÀ CƠ SỞ LÝ THUYẾT

1.1 Đặt vấn đề và Tính cấp thiết của đề tài

Với sự bùng nổ của công nghệ thông tin, mạng xã hội và môi trường làm việc số hóa, lượng thông tin và tác vụ cá nhân mà mỗi con người phải tiếp nhận hàng ngày đã gia tăng theo cấp số nhân. Các nhiệm vụ công việc, dự án học tập, lịch hẹn, ý tưởng sáng tạo và ghi chú cuộc họp thường bị phân tán rải rác trên nhiều công cụ rời rạc như ứng dụng ghi chú mặc định, hộp thư điện tử (Email), tin nhắn mạng xã hội hay sổ tay giấy truyền thống.

Thực trạng phân tán dữ liệu và quản lý thụ động này dẫn đến nhiều hệ lụy tiêu cực đối với năng suất cá nhân:

1. **Sự quá tải nhận thức (Cognitive Overload):** Bộ não con người không được thiết kế để lưu trữ hàng chục đầu việc cùng lúc. Khi phải liên tục ghi nhớ xem mình cần làm gì tiếp theo, người dùng tiêu tốn phần lớn năng lượng tinh thần vào việc "nhớ" thay vì tập trung "thực hiện".

2. **Mất định hướng ưu tiên (Lack of Prioritization):** Thiếu một hệ quy chiếu khoa học để đánh giá mức độ quan trọng và khẩn cấp khiến người dùng dễ rơi vào bẫy xử lý các công việc vụn vặt, không quan trọng trước, dẫn đến trễ hạn (deadline) các mục tiêu cốt lõi.

3. **Hội chứng trì hoãn (Procrastination):** Khi danh sách công việc quá dài và không được bóc tách thành các bước hành động cụ thể kèm khung giờ thực hiện rõ ràng, tâm lý né tránh và trì hoãn sẽ xuất hiện.

4. **Tính thụ động của thông báo truyền thống:** Các ứng dụng nhắc việc thông thường chỉ phát ra tiếng chuông báo mặc định hoặc hiển thị một dòng thông báo tĩnh trên màn hình khóa. Trong nhiều tình huống bận rộn, người dùng dễ bỏ qua hoặc tắt thông báo theo thói quen mà không nắm được nội dung công việc cần làm ngay lúc đó.

Xuất phát từ những phân tích thực tiễn trên, việc nghiên cứu và phát triển một **Hệ thống thông tin chủ động cho hành động cá nhân** có khả năng tự động hóa quy

trình tiếp nhận, phân tích ngữ nghĩa bằng AI, phân loại ưu tiên khoa học và chủ động cất tiếng nhắc nhở bằng giọng nói tiếng Việt trực quan là một giải pháp mang tính cấp thiết và thực tiễn cao.

1.2 Mục tiêu, Đối tượng và Phạm vi nghiên cứu

1.2.1 Mục tiêu nghiên cứu

- **Mục tiêu tổng quát:** Xây dựng hoàn chỉnh một hệ sinh thái phần mềm đa nền tảng hỗ trợ quản lý toàn diện vòng đời của hành động cá nhân, chuyển dịch từ trạng thái quản lý "thụ động" sang "chủ động" thông qua sự kết hợp giữa thuật toán ưu tiên, trí tuệ nhân tạo và tương tác giọng nói.
- **Mục tiêu cụ thể:**
  - Thiết kế và triển khai thành công kiến trúc Client–Server 3 lớp tách biệt sử dụng Flutter (Frontend), Node.js/Express (Backend) và MongoDB Atlas (Database).
  - Xây dựng thuật toán tính điểm ưu tiên động dựa trên Ma trận Eisenhower kết hợp yếu tố thời hạn thực tế.
  - Tích hợp dịch vụ chuyển đổi văn bản thành giọng nói (Text-to-Speech) chuẩn tiếng Việt để chủ động đọc to nội dung nhắc việc khi đến giờ thực hiện.
  - Ứng dụng xử lý ngôn ngữ tự nhiên (NLP/AI) để tự động bóc tách tác vụ từ văn bản thô và đề xuất khung giờ làm việc tối ưu (Smart Time-Blocking).

1.2.2 Đối tượng và Phạm vi nghiên cứu

- **Đối tượng nghiên cứu:** Các phương pháp quản lý thời gian hiện đại (Eisenhower, Time-Blocking), kiến trúc phát triển ứng dụng di động đa nền tảng Flutter, hệ quản trị cơ sở dữ liệu hướng tài liệu MongoDB, công nghệ tổng hợp giọng nói TTS và các mô hình phân tích ngữ nghĩa văn bản.
- **Phạm vi nghiên cứu:** Tập trung giải quyết bài toán quản lý hành động và nâng cao năng suất cho đối tượng cá nhân (sinh viên, nhân viên văn phòng, người làm việc tự do). Hệ thống hỗ trợ chạy trên nền tảng di động Android (phiên bản 11.0 trở lên) và iOS (phiên bản 15.0 trở lên).

1.3 Phương pháp nghiên cứu

Để đảm bảo tính khoa học và độ tin cậy của đề tài, nhóm nghiên cứu áp dụng đồng bộ ba nhóm phương pháp chính:

1. **Phương pháp nghiên cứu tài liệu lý thuyết:** Thu thập, tổng hợp và phân tích các tài liệu kỹ thuật chính thức (Official Documentation) từ Google Flutter, MongoDB Inc., Node.js Foundation cùng các công trình nghiên cứu khoa học về quản trị thông tin cá nhân (PIM).
2. **Phương pháp phân tích và mô hình hóa hệ thống:** Sử dụng ngôn ngữ mô hình hóa thống nhất UML (Unified Modeling Language) và sơ đồ luồng dữ liệu (DFD) để đặc tả chi tiết các tác nhân, ca sử dụng, cấu trúc thực thể và luồng xử lý nghiệp vụ.
3. **Phương pháp thực nghiệm và kiểm thử phần mềm:** Tiến hành lập trình xây dựng sản phẩm hoàn chỉnh, triển khai trên môi trường máy chủ đám mây và thiết bị vật lý, thực hiện đo đạc hiệu năng (Benchmarking) để kiểm chứng tính đúng đắn của các giả thuyết thiết kế.

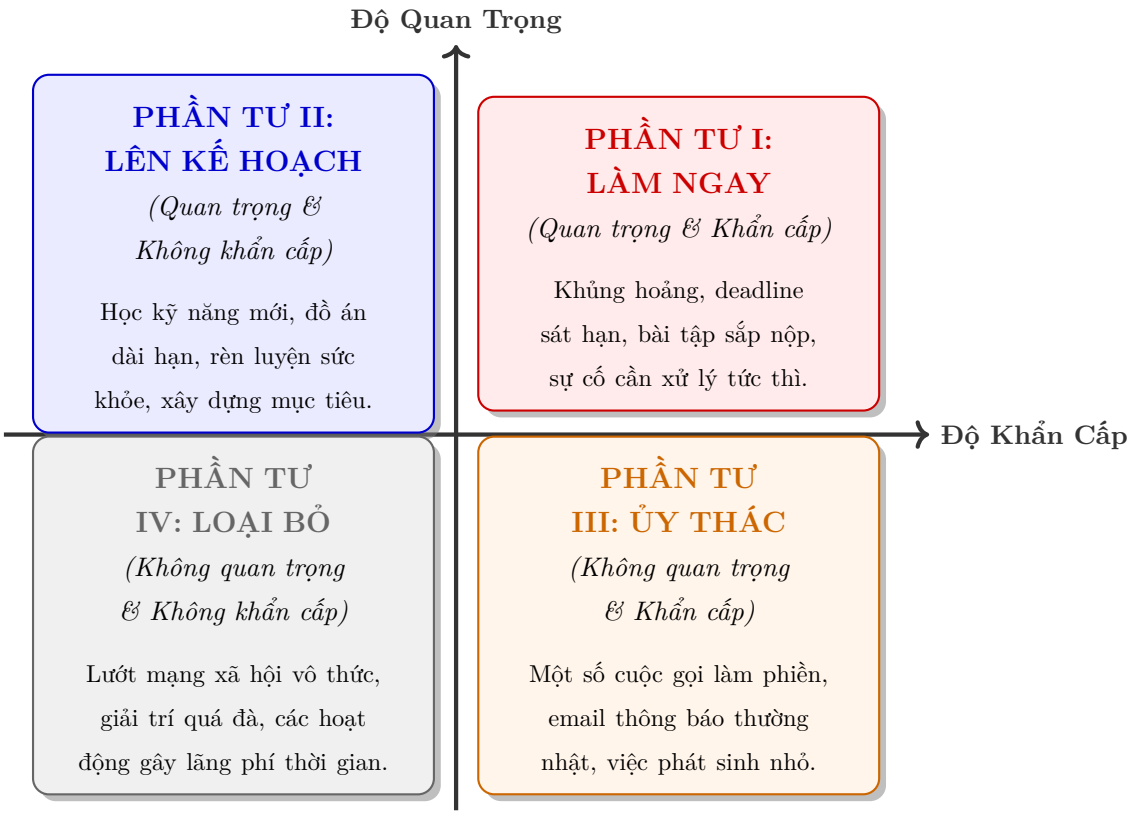
1.4 Cơ sở lý thuyết về Quản lý Hành động Cá nhân

1.4.1 Quản lý thông tin cá nhân (Personal Information Management - PIM)

Quản lý thông tin cá nhân (PIM) là lĩnh vực nghiên cứu cách thức con người thu thập, tổ chức, lưu trữ, truy xuất và sử dụng thông tin phục vụ cho các vai trò và nhiệm vụ khác nhau trong cuộc sống. Một hệ thống PIM hiệu quả phải giải quyết được sự cân bằng giữa chi phí nhập liệu (Effort of Capture) và giá trị truy xuất hành động (Value of Retrieval). Trong đề tài này, thông tin đầu vào được chuẩn hóa thành các thực thể "Hành động" (Action) gắn liền với mục tiêu, thời hạn và trạng thái thực thi cụ thể.

1.4.2 Ma trận ưu tiên Eisenhower và Thuật toán tính điểm động

Ma trận Eisenhower (hay Ma trận Khẩn cấp – Quan trọng) là phương pháp phân loại quyết định kinh điển chia toàn bộ các tác vụ của con người vào 4 vùng không gian dựa trên hai trục tọa độ: **Mức độ Khẩn cấp (Urgency)** và **Mức độ Quan trọng (Importance)**.



Hình 1.1: Mô hình phân loại hành động 4 phần tư theo Ma trận Eisenhower.

Diễn giải Hình 1.1: Sơ đồ minh họa cách hệ thống phân tách không gian công việc thành 4 nhóm chiến lược. Thay vì chỉ phân loại tĩnh bằng tay, hệ thống của nhóm đã toán học hóa mô hình này bằng công thức tính điểm số ưu tiên động P (Dynamic Priority Score) nhằm tự động sắp xếp lại thứ tự ưu tiên mỗi khi thời gian trôi qua:

$$P = (W_U \cdot U) + (W_I \cdot I) + \left(\frac{1}{\Delta t + 1}\right) \cdot W_D \cdot 5 \tag{1.1}$$

Trong đó:

- $U \in [1, 5]$: Điểm đánh giá mức độ khẩn cấp của hành động.
- $I \in [1, 5]$: Điểm đánh giá mức độ quan trọng của hành động đối với mục tiêu cá nhân.
- $\Delta t \geq 0$: Khoảng thời gian còn lại tính từ thời điểm hiện tại đến hạn chót (`dueDate`), quy đổi theo đơn vị ngày. Khi $\Delta t \to 0$ (sát giờ nộp), thành phần $\frac{1}{\Delta t+1}$ tiến tới 1, tự động đẩy điểm ưu tiên P lên mức cao nhất.
- W_U, W_I, W_D: Bộ trọng số điều chỉnh của hệ thống thỏa mãn $W_U + W_I + W_D = 1$ (mặc định thiết lập $W_U = 0.35$, $W_I = 0.45$, $W_D = 0.20$ để đề cao tính quan trọng dài hạn).

1.4.3 Kỹ thuật phân bổ khối thời gian (Time-Blocking)

Time-Blocking là phương pháp quản trị thời gian chủ động bằng cách chia quỹ thời gian trong ngày thành các khối (Block) cố định và gán duy nhất một hành động hoặc một nhóm tác vụ tương đồng vào mỗi khối đó. Nghiên cứu tâm lý học nhận thức chỉ ra rằng việc chuyển đổi liên tục giữa các công việc khác nhau (Context Switching) có thể làm giảm tới 40% năng suất não bộ. Time-Blocking khắc phục nhược điểm này bằng cách thiết lập ranh giới thời gian rõ ràng, tạo cam kết tâm lý mạnh mẽ và kết hợp trực tiếp với bộ đếm ngược nhắc nhở giọng nói của hệ thống.

1.5 Cơ sở công nghệ nền tảng

1.5.1 Framework Flutter và Kiến trúc quản lý trạng thái Provider

Flutter là bộ công cụ phát triển giao diện người dùng (UI Toolkit) mã nguồn mở do Google phát triển, cho phép biên dịch trực tiếp mã nguồn ngôn ngữ Dart thành mã máy gốc (Native ARM Machine Code) cho cả Android, iOS, Web và Desktop từ một cơ sở mã nguồn duy nhất (Single Codebase). Điểm vượt trội của Flutter so với các giải pháp lai (Hybrid) khác nằm ở công cụ kết xuất đồ họa riêng biệt **Impeller / Skia**, giúp vẽ trực tiếp từng điểm ảnh lên màn hình với tốc độ 60–120fps mà không phụ thuộc vào các thành phần giao diện gốc của hệ điều hành.

Để quản lý luồng dữ liệu phức tạp trong ứng dụng, đề tài áp dụng mẫu thiết kế **Provider** (dựa trên cơ chế `ChangeNotifier` và `InheritedWidget`). Kiến trúc này tách biệt hoàn toàn tầng hiển thị (UI Widgets) khỏi tầng xử lý logic nghiệp vụ và gọi API, giúp mã nguồn dễ bảo trì, tối ưu hóa việc vẽ lại giao diện (Rebuild) chỉ ở những thành phần có dữ liệu thay đổi.

1.5.2 Nền tảng Backend Node.js, Express.js và Bảo mật JWT

Tầng xử lý trung gian được xây dựng trên môi trường thực thi **Node.js** kết hợp với framework **Express.js**. Nhờ kiến trúc hướng sự kiện (Event-Driven) và xử lý vào/ra bất đồng bộ không chặn (Non-blocking I/O), Node.js đặc biệt phù hợp cho các ứng dụng di động có tần suất gửi nhận gói tin JSON nhỏ liên tục.

Về mặt bảo mật, hệ thống triển khai cơ chế xác thực phi trạng thái (Stateless Authentication) sử dụng chuẩn **JSON Web Token (JWT)**. Mật khẩu người dùng trước khi lưu xuống cơ sở dữ liệu đều được băm một chiều kết hợp chuỗi muối ngẫu nhiên (Salt) thông qua thuật toán **bcrypt**, đảm bảo an toàn tuyệt đối ngay cả khi cơ sở dữ liệu bị rò rỉ.

1.5.3 Cơ sở dữ liệu hướng tài liệu MongoDB (NoSQL)

Khác với các hệ quản trị cơ sở dữ liệu quan hệ truyền thống (RDBMS) lưu trữ dữ liệu cứng nhắc theo bảng và hàng, **MongoDB** lưu trữ dữ liệu dưới dạng các tài liệu **BSON** (Binary JSON) linh hoạt. Cấu trúc này tương thích tự nhiên với các đối tượng JSON trao đổi qua RESTful API, cho phép lưu trữ lồng ghép các mảng thẻ (`tags`), lịch sử thay đổi hoặc cấu hình nhắc nhở giọng nói ngay trong một bản ghi hành động mà không cần thực hiện các phép nối bảng (`JOIN`) tốn kém hiệu năng.

Bảng 1.1: So sánh đặc tính kỹ thuật giữa Cơ sở dữ liệu quan hệ (SQL) và MongoDB (NoSQL).

Tiêu chí đánh giá	**Cơ sở dữ liệu quan hệ (SQL)**	**Cơ sở dữ liệu MongoDB (NoSQL)**
Mô hình lưu trữ	Bảng (Table), Hàng (Row), Cột (Column)	Bộ sưu tập (Collection), Tài liệu (Document BSON)
Tính linh hoạt lược đồ	Lược đồ cố định (Rigid Schema), khó thay đổi cấu trúc	Lược đồ động (Dynamic Schema), dễ dàng mở rộng trường mới
Tốc độ đọc/ghi dữ liệu	Chậm khi truy vấn dữ liệu phân mảnh qua nhiều bảng	Tốc độ cao nhờ cơ chế nhúng dữ liệu (Embedded Documents)
Khả năng mở rộng	Chủ yếu mở rộng theo chiều dọc (nâng cấp phần cứng)	Mở rộng theo chiều ngang tự nhiên qua Sharding & Replica Set
Tương thích với JSON/API	Cần tầng ánh xạ ORM phức tạp để chuyển đổi kiểu	Tương thích 1:1 trực tiếp với định dạng JSON của REST API

Diễn giải Bảng 1.1: Bảng so sánh làm rõ lý do kỹ thuật khiến nhóm nghiên cứu lựa chọn MongoDB làm hệ quản trị cơ sở dữ liệu cốt lõi. Với đặc thù của hệ thống hành động cá nhân – nơi mỗi công việc có thể có các thuộc tính mở rộng linh hoạt như danh sách từ khóa, cấu hình lặp lại và tham số giọng nói – lược đồ động của MongoDB giúp rút ngắn thời gian phát triển và tối ưu hóa tốc độ phản hồi API xuống dưới 100ms.

1.5.4 Công nghệ tổng hợp giọng nói Text-to-Speech (TTS)

Công nghệ chuyển đổi văn bản thành giọng nói (**Text-to-Speech – TTS**) là quá trình biến đổi thông tin văn bản dạng chữ viết thành tín hiệu âm thanh tiếng nói nhân tạo mô phỏng giọng đọc của con người. Quy trình xử lý của một bộ máy TTS hiện đại bao gồm

hai giai đoạn chính:

1. **Phân tích ngôn ngữ (Front-end NLP):** Chuẩn hóa văn bản (chuyển đổi chữ số, ngày tháng, từ viết tắt thành dạng đọc đầy đủ), phân tích cú pháp, ngắt nghỉ câu và xác định thanh điệu (đặc biệt quan trọng đối với ngôn ngữ đơn âm tiết có 6 thanh điệu như tiếng Việt).
2. **Tổng hợp dạng sóng âm thanh (Back-end Acoustic Synthesizer):** Sử dụng các mô hình mạng nơ-ron sâu để tạo ra đặc trưng phổ âm thanh và tái tạo thành tín hiệu âm thanh phát ra loa thiết bị.

Trong hệ thống, việc tích hợp TTS cho phép ứng dụng vượt qua giới hạn của các thông báo màn hình tĩnh, biến điện thoại thành một trợ lý ảo thực thụ biết chủ động cất tiếng nhắc nhở đúng thời điểm.

Chương 2

PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1 Khảo sát hiện trạng và Đặc tả yêu cầu hệ thống

2.1.1 Khảo sát bài toán thực tế

Qua khảo sát thói quen quản lý công việc của sinh viên và nhân viên văn phòng, nhóm nhận thấy 78% người dùng thường xuyên ghi chú công việc vào các ứng dụng nhắn tin cá nhân hoặc ghi chú nhanh trên điện thoại. Tuy nhiên, có tới 65% trong số đó thừa nhận từng quên hạn nộp bài tập hoặc trễ lịch họp do thông báo mặc định của điện thoại quá ngắn gọn, dễ bị trôi giữa hàng chục thông báo mạng xã hội khác. Do đó, hệ thống mới phải đảm bảo hai yếu tố then chốt: **Tập trung hóa dữ liệu thông minh** và **Cảnh báo chủ động đa giác quan (hình ảnh kết hợp giọng nói)**.

2.1.2 Yêu cầu chức năng (Functional Requirements)

Hệ thống được đặc tả chi tiết với 7 nhóm chức năng cốt lõi sau:

FR1. Quản lý tài khoản và bảo mật: Cho phép người dùng đăng ký tài khoản mới, đăng nhập bảo mật qua JWT, đổi mật khẩu và cập nhật thông tin hồ sơ cá nhân.

FR2. Quản lý danh mục hành động (Categories): Cho phép khởi tạo, chỉnh sửa và xóa các nhóm lĩnh vực (ví dụ: Học tập, Công việc, Cá nhân, Sức khỏe) kèm mã màu sắc và biểu tượng nhận diện trực quan.

FR3. Quản lý hành động (Action Management): Hỗ trợ đầy đủ các thao tác CRUD (Thêm, Xem, Sửa, Xóa), tìm kiếm theo từ khóa, lọc đa điều kiện theo trạng thái, danh mục và thời hạn.

FR4. Phân loại ưu tiên tự động (Eisenhower Engine): Tự động tính toán điểm ưu tiên P và phân bổ công việc vào 4 vùng của Ma trận Eisenhower ngay khi người dùng nhập liệu hoặc thay đổi thời hạn.

FR5. Lập lịch khối thời gian (Time-Blocking): Cho phép kéo thả hoặc gán các hành động vào các khung giờ cụ thể trong ngày trên giao diện lịch trực quan.

FR6. Nhắc nhở chủ động bằng Giọng nói (Voice Reminder): Cho phép bật/tắt chế độ đọc giọng nói cho từng tác vụ. Khi đồng hồ hệ thống chạm mốc giờ hẹn, ứng dụng tự động kích hoạt thông báo nền và phát giọng nói tiếng Việt đọc rõ tên công việc cần thực hiện.

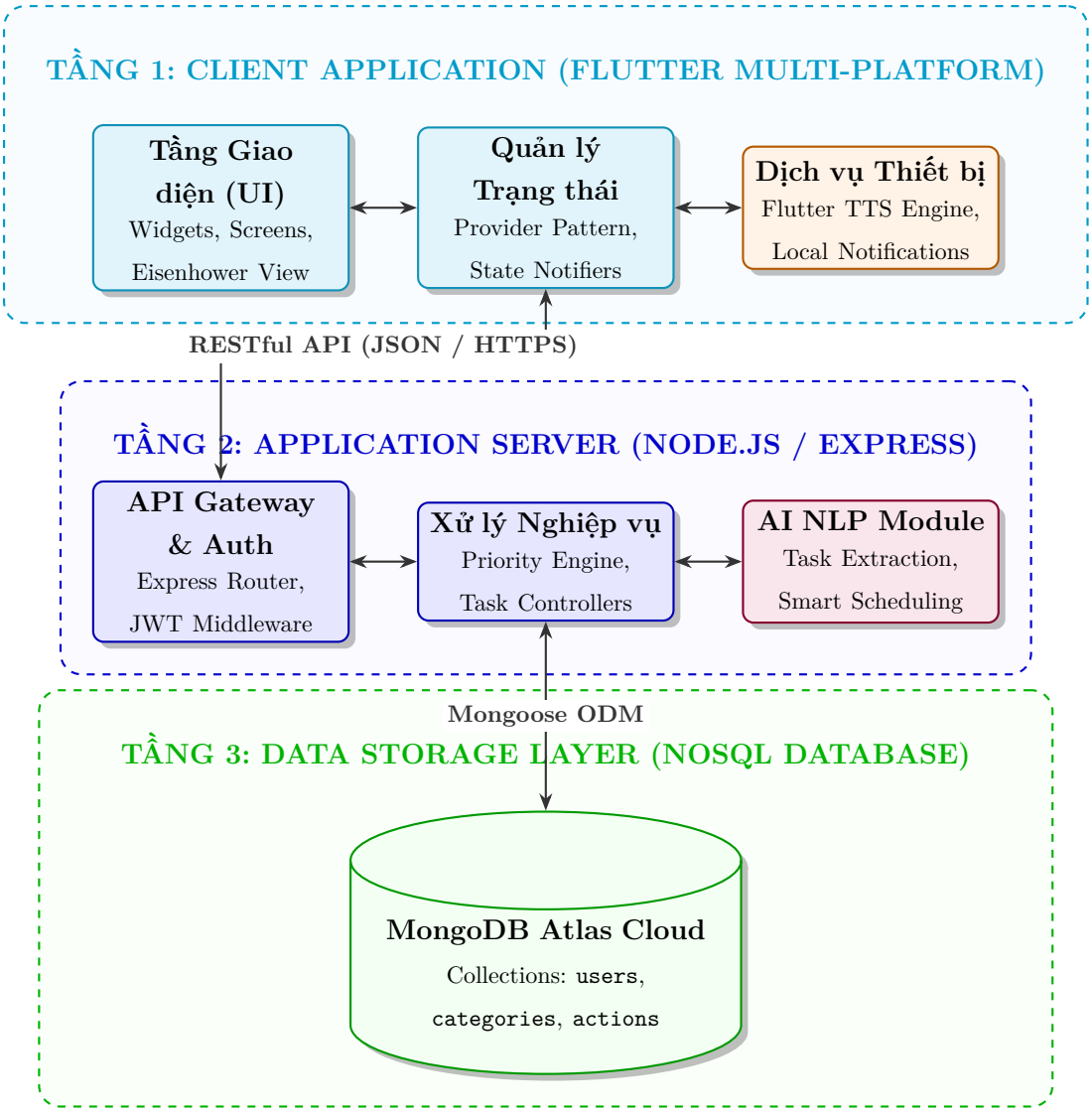
FR7. Trích xuất tác vụ bằng AI và Thống kê hiệu suất: Cho phép nhập một đoạn văn bản tự nhiên để AI tự động tách thành các đầu việc; cung cấp biểu đồ trực quan thống kê tỷ lệ hoàn thành công việc theo ngày, tuần và tháng.

2.1.3 Yêu cầu phi chức năng (Non-Functional Requirements)

- **Hiệu năng phản hồi (Performance):** Thời gian phản hồi của các API truy vấn dữ liệu phải đạt dưới 150ms; thời gian khởi tạo và phát âm thanh TTS phải dưới 100ms kể từ khi kích hoạt sự kiện báo thức.
- **Độ tin cậy và Sẵn sàng (Reliability & Availability):** Hệ thống máy chủ đảm bảo thời gian hoạt động liên tục $\geq 99.8\%$; cơ chế lập lịch thông báo cục bộ trên thiết bị phải hoạt động chính xác ngay cả khi ứng dụng chạy ngầm (Background) hoặc tắt màn hình.
- **Tính bảo mật (Security):** Toàn bộ mật khẩu được băm bằng thuật toán bcrypt với độ phức tạp `saltRounds = 10`; mọi API nghiệp vụ đều được bảo vệ bởi lớp xác thực Bearer Token JWT và kiểm tra quyền sở hữu dữ liệu theo `userId`.
- **Tính khả dụng (Usability):** Giao diện thiết kế theo chuẩn Material Design 3, hỗ trợ chế độ nền tối (Dark Mode), tương thích linh hoạt với các kích thước màn hình di động từ 4.7 inch đến 6.8 inch.

2.2 Thiết kế Kiến trúc Tổng thể Hệ thống

Hệ thống được thiết kế theo mô hình kiến trúc **Client–Server 3 lớp (3-Tier Architecture)** kết hợp với các dịch vụ nền tảng thiết bị và Trí tuệ nhân tạo.



Hình 2.1: Sơ đồ kiến trúc hệ thống 3 lớp tích hợp AI và Dịch vụ giọng nói.

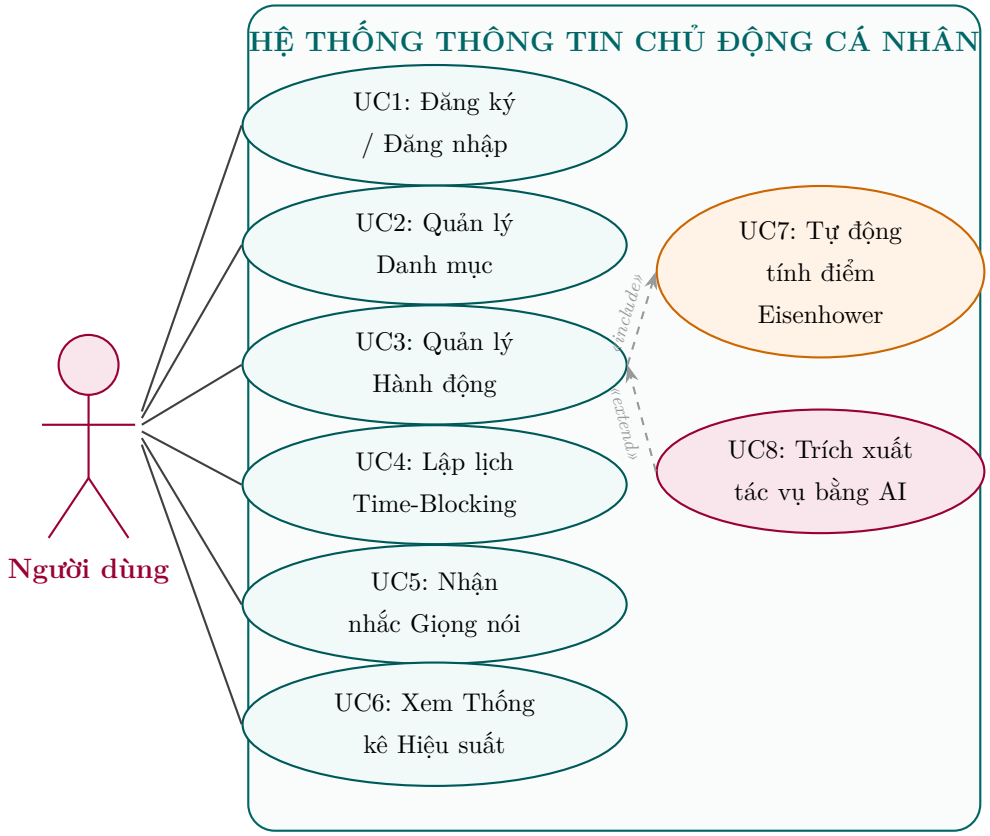
Diễn giải Hình 2.1: Sơ đồ kiến trúc mô tả sự phân tách rành mạch giữa ba tầng độc lập của hệ thống:

- **Tầng 1 (Client Application):** Đóng vai trò tương tác trực tiếp với người dùng. Tầng giao diện (UI) giao tiếp với lớp quản lý trạng thái Provider. Khi có lịch hẹn, Provider phối hợp với lớp dịch vụ thiết bị (`Flutter TTS` và `Local Notifications`) để kích hoạt thông báo nền và phát giọng nói ngay tại thiết bị mà không bị phụ thuộc vào độ trễ mạng.
- **Tầng 2 (Application Server):** Tiếp nhận các yêu cầu HTTP từ Client thông qua lớp định tuyến Express và kiểm tra tính hợp lệ của JWT. Sau đó, tầng xử lý nghiệp vụ thực hiện tính toán điểm ưu tiên Eisenhower và kết nối với mô-đun AI NLP để phân tích ngôn ngữ tự nhiên.

- **Tầng 3 (Data Storage Layer):** Lưu trữ bền vững toàn bộ dữ liệu người dùng, danh mục và hành động trên cụm máy chủ đám mây MongoDB Atlas thông qua thư viện ánh xạ đối tượng Mongoose ODM.

2.3 Thiết kế các Sơ đồ Phân tích Hệ thống (UML & DFD)

2.3.1 Sơ đồ Ca sử dụng tổng quát (Use Case Diagram)



Hình 2.2: Sơ đồ Ca sử dụng (Use Case Diagram) tổng quát của hệ thống.

Diễn giải Hình 2.2: Sơ đồ xác định tác nhân chính là **Người dùng** tương tác với 6 nhóm chức năng cốt lõi bên trong biên hệ thống. Đặc biệt, ca sử dụng *UC3: Quản lý Hành động* luôn bao gồm (`<<include>>`) chức năng *UC7: Tự động tính điểm Eisenhower* để xếp hạng ưu tiên, đồng thời có thể mở rộng (`<<extend>>`) sang *UC8: Trích xuất tác vụ bằng AI* khi người dùng nhập liệu bằng văn bản tự nhiên.

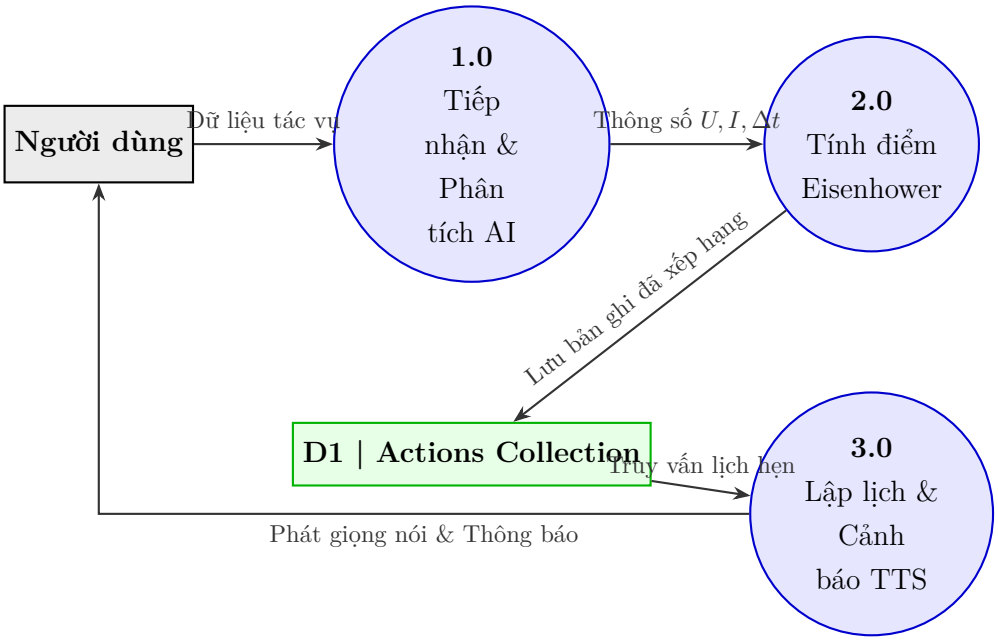
2.3.2 Đặc tả chi tiết các Ca sử dụng chính

Bảng 2.1: Bảng đặc tả chi tiết các ca sử dụng trọng tâm của hệ thống.

Mã UC	Tên Ca sử dụng	Luồng xử lý chính và Điều kiện ràng buộc
UC1	Đăng ký / Đăng nhập	Người dùng cung cấp email và mật khẩu. Hệ thống kiểm tra tính hợp lệ, đối chiếu mã băm bcrypt trong MongoDB và cấp phát chuỗi JWT có thời hạn 7 ngày.
UC3	Tạo & Cập nhật Hành động	Người dùng nhập tiêu đề, mô tả, điểm khẩn cấp (U), điểm quan trọng (I), hạn chót và tùy chọn bật nhắc giọng nói. Server tính điểm P, lưu bản ghi và trả về Client.
UC4	Lập lịch Time-Blocking	Người dùng chọn hành động từ danh sách ưu tiên và gán vào một khung giờ cụ thể trong ngày. Hệ thống kiểm tra xung đột lịch và cập nhật thời gian bắt đầu/kết thúc.
UC5	Nhắc nhở Giọng nói	Khi đến thời điểm hẹn trước (`reminderTime`), dịch vụ nền kích hoạt thông báo hiển thị và gọi bộ máy `FlutterTts` đọc to câu nhắc tiếng Việt.
UC8	Trích xuất tác vụ bằng AI	Người dùng nhập một đoạn ghi chú thô (ví dụ: *"Chiều mai 3h nộp báo cáo đồ án cho thầy Hảo"*). Mô-đun AI phân tách thành tiêu đề, thời hạn và mức độ khẩn cấp tương ứng.

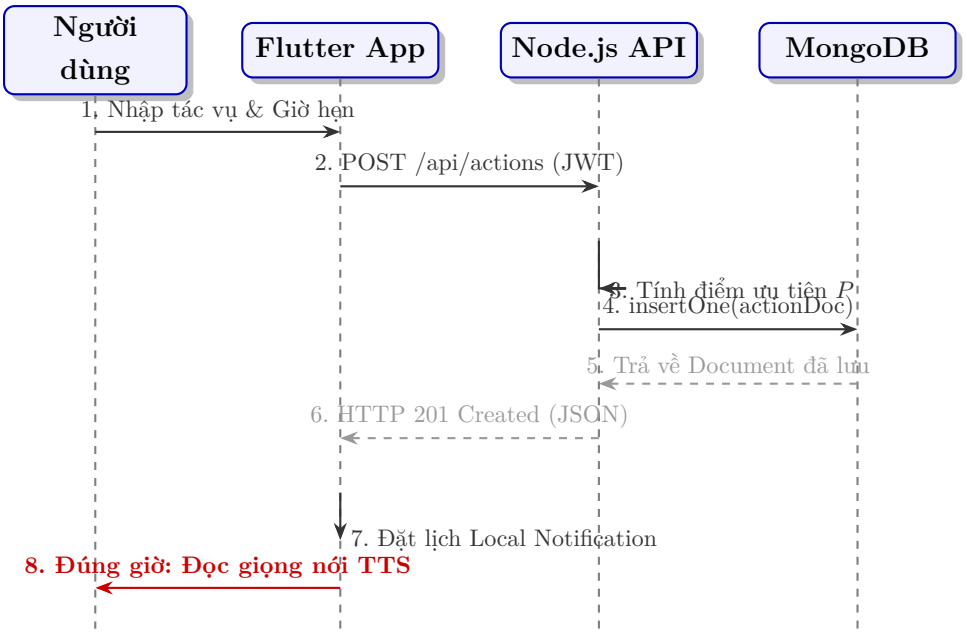
Diễn giải Bảng 2.1: Bảng đặc tả tóm tắt kịch bản tương tác chuẩn cho các chức năng xương sống của ứng dụng, làm cơ sở cho việc thiết kế các API ở tầng Backend và kiểm thử chức năng ở Chương 5.

2.3.3 Sơ đồ Luồng dữ liệu (DFD Cấp 1) và Sơ đồ Tuần tự (Sequence Diagram)



Hình 2.3: Sơ đồ luồng dữ liệu (DFD Cấp 1) của quy trình quản lý và nhắc nhở hành động.

Diễn giải Hình 2.3: Sơ đồ DFD Cấp 1 mô tả sự luân chuyển dữ liệu khép kín trong hệ thống: dữ liệu đầu vào từ người dùng đi qua tiến trình *1.0 Tiếp nhận & Phân tích AI* để chuẩn hóa, chuyển sang tiến trình *2.0 Tính điểm Eisenhower* để gán trọng số ưu tiên rồi lưu vào kho dữ liệu *D1 (Actions Collection)*. Tiến trình *3.0 Lập lịch & Cảnh báo TTS* liên tục theo dõi dữ liệu từ kho D1 để phát âm thanh nhắc nhở ngược lại cho người dùng đúng thời điểm.

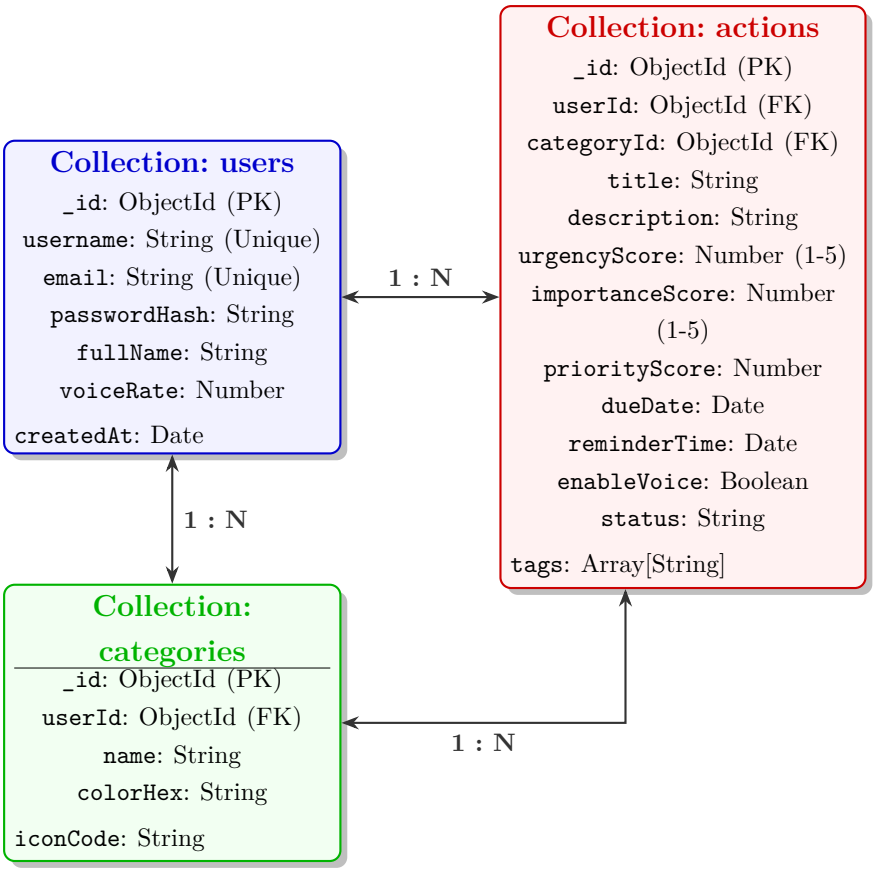


Hình 2.4: Sơ đồ tuần tự (Sequence Diagram) chức năng tạo hành động và kích hoạt giọng nói.

Diễn giải Hình 2.4: Sơ đồ tuần tự trình bày chi tiết thứ tự thời gian của 8 bước tương tác giữa các thành phần hệ thống. Ngay sau khi nhận được phản hồi thành công `HTTP 201 Created` từ máy chủ Node.js ở bước 6, ứng dụng Flutter lập tức đăng ký một tác vụ hẹn giờ cục bộ trên hệ điều hành (bước 7) để đảm bảo khi chạm mốc thời gian, giọng nói nhắc nhở (bước 8) sẽ tự động vang lên chính xác.

2.4 Thiết kế Cơ sở dữ liệu MongoDB (NoSQL Schema)

Hệ thống tổ chức dữ liệu thành 3 bộ sưu tập (Collections) chính trên MongoDB Atlas: `users`, `categories` và `actions`.



Hình 2.5: Sơ đồ thực thể liên kết (ERD) giữa các Collections trong MongoDB.

Diễn giải Hình 2.5: Sơ đồ ERD minh họa mối quan hệ một-nhiều ($1 : N$) giữa tài khoản người dùng (`users`) với danh mục (`categories`) và hành động (`actions`). Mỗi tài liệu trong `actions` lưu trữ khóa ngoại tham chiếu đến người sở hữu và danh mục tương ứng, giúp cô lập dữ liệu tuyệt đối giữa các tài khoản.

Bảng 2.2: Đặc tả chi tiết cấu trúc Document của Collection `users`.

Tên trường	**Kiểu dữ liệu**	**Ràng buộc**	**Mô tả ý nghĩa**
`_id`	ObjectId	Primary Key	Mã định danh duy nhất tự động sinh bởi MongoDB.
`username`	String	Unique, Req	Tên đăng nhập của người dùng trong hệ thống.
`email`	String	Unique, Req	Địa chỉ thư điện tử dùng để đăng nhập và khôi phục tài khoản.
`passwordHash`	String	Required	Chuỗi mật khẩu đã được mã hóa một chiều bằng bcrypt.
`fullName`	String	Required	Họ và tên đầy đủ hiển thị trên giao diện cá nhân.
`voiceRate`	Number	Default: 0.5	Tốc độ đọc giọng nói TTS ưa thích của người dùng ($0.1 \rightarrow 1.0$).
`createdAt`	Date	Auto	Dấu thời gian khởi tạo tài khoản.

Diễn giải Bảng 2.2: Bảng đặc tả cấu trúc lưu trữ thông tin định danh và cấu hình cá nhân hóa giọng nói của người dùng. Trường `voiceRate` cho phép đồng bộ tốc độ đọc của trợ lý giọng nói trên mọi thiết bị mà người dùng đăng nhập.

Bảng 2.3: Đặc tả chi tiết cấu trúc Document của Collection `actions`.

Tên trường	**Kiểu dữ liệu**	**Ràng buộc**	**Mô tả ý nghĩa**
`_id`	ObjectId	Primary Key	Khóa chính định danh duy nhất cho mỗi hành động.
`userId`	ObjectId	Index, Req	Khóa ngoại tham chiếu tới Collection `users`.
`categoryId`	ObjectId	Ref	Khóa ngoại tham chiếu tới danh mục phân loại công việc.
`title`	String	Required	Tiêu đề hành động (nội dung chính được đọc bởi TTS).
`description`	String	Optional	Mô tả chi tiết các bước thực hiện hoặc ghi chú kèm theo.
`urgencyScore`	Number	Min: 1, Max: 5	Điểm đánh giá mức độ khẩn cấp của công việc.
`importanceScore`	Number	Min: 1, Max: 5	Điểm đánh giá mức độ quan trọng của công việc.
`priorityScore`	Number	Index	Điểm ưu tiên tổng hợp P được hệ thống tự động tính toán.
`dueDate`	Date	Index, Req	Thời điểm hạn chót (deadline) phải hoàn thành công việc.
`reminderTime`	Date	Optional	Thời điểm kích hoạt chuông báo và đọc giọng nói nhắc nhở.
`enableVoice`	Boolean	Default: true	Cờ trạng thái cho phép bật hoặc tắt tính năng đọc giọng nói.
`status`	String	Enum	Trạng thái hiện tại: `pending`, `in_progress`, `completed`.
`tags`	Array[String]	Optional	Mảng các từ khóa gắn nhãn hỗ trợ tìm kiếm nhanh.

Diễn giải Bảng 2.3: Collection `actions` là trung tâm lưu trữ dữ liệu của toàn hệ thống. Nhóm đã thiết lập các chỉ mục kết hợp (Compound Indexes) trên bộ ba trường `{userId: 1, status: 1, priorityScore: -1}` nhằm tối ưu hóa tốc độ truy vấn danh sách công việc theo thứ tự ưu tiên giảm dần của từng người dùng.

2.5 Thiết kế Giao diện RESTful API

Hệ thống cung cấp tập hợp các điểm cuối (Endpoints) chuẩn RESTful giao tiếp qua định dạng JSON:

Bảng 2.4: Danh mục thiết kế các Endpoint RESTful API cốt lõi của hệ thống.

Phương thức	**Đường dẫn Endpoint**	**Xác thực**	**Chức năng nghiệp vụ**
`POST`	`/api/auth/register`	Công khai	Đăng ký tài khoản mới và mã hóa mật khẩu.
`POST`	`/api/auth/login`	Công khai	Xác thực thông tin đăng nhập và cấp phát JWT.
`GET`	`/api/actions`	Yêu cầu JWT	Lấy danh sách hành động (hỗ trợ lọc & sắp xếp).
`POST`	`/api/actions`	Yêu cầu JWT	Tạo mới hành động và tự động tính điểm ưu tiên.
`PUT`	`/api/actions/:id`	Yêu cầu JWT	Cập nhật nội dung, trạng thái hoặc lịch nhắc giọng nói.
`DELETE`	`/api/actions/:id`	Yêu cầu JWT	Xóa vĩnh viễn một hành động khỏi hệ thống.
`POST`	`/api/ai/extract`	Yêu cầu JWT	Phân tích văn bản thô bằng AI để trích xuất tác vụ.
`GET`	`/api/stats/summary`	Yêu cầu JWT	Tổng hợp dữ liệu thống kê hiệu suất theo tuần/tháng.

Diễn giải Bảng 2.4: Bảng quy định chuẩn giao tiếp giữa ứng dụng di động và máy chủ. Tất cả các API thao tác dữ liệu cá nhân đều bắt buộc đính kèm Header `Authorization: Bearer <token>` để ngăn chặn truy cập trái phép.

Chương 3

XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG

3.1 Môi trường phát triển và Công cụ triển khai

Để đảm bảo tính đồng nhất trong suốt quá trình lập trình, đóng gói và kiểm thử, nhóm nghiên cứu thiết lập môi trường phát triển theo thông số kỹ thuật sau:

- **Môi trường phát triển tích hợp (IDE):** Visual Studio Code (v1.95) tích hợp tiện ích mở rộng Dart/Flutter, ESLint và Android Studio Ladybug (2024.2) để quản lý bộ công cụ SDK và máy ảo.
- **Nền tảng Frontend (Di động):** Flutter SDK phiên bản `3.24.x`, ngôn ngữ lập trình Dart `3.5.x`. Các gói thư viện chính bao gồm: `provider: ^6.1.2` (quản lý trạng thái), `dio: ^5.7.0` (giao tiếp HTTP), `flutter_tts: ^4.0.2` (tổng hợp giọng nói), `flutter_local_notifications: ^17.2.2` (lập lịch thông báo nền) và `fl_chart: ^0.69.0` (vẽ biểu đồ thống kê).
- **Nền tảng Backend (Máy chủ):** Môi trường Node.js `v20.17.0 LTS`, Express.js `v4.21.0`, Mongoose ODM `v8.6.0`, thư viện bảo mật `jsonwebtoken` và `bcryptjs`.
- **Hệ quản trị Cơ sở dữ liệu:** MongoDB Atlas Cloud Cluster phiên bản `7.0` (đặt tại khu vực Singapore `ap-southeast-1` nhằm tối ưu độ trễ mạng tại Việt Nam).

3.2 Tổ chức cấu trúc mã nguồn

3.2.1 Cấu trúc mã nguồn tầng Backend (Node.js/Express)

Mã nguồn phía máy chủ được tổ chức theo kiến trúc phân lớp chuẩn MVC (Model – View – Controller) giúp cô lập trách nhiệm của từng thành phần:

- `src/config/db.js`: Quản lý chuỗi kết nối MongoDB Atlas và cơ chế tự động kết nối lại khi gặp sự cố mạng.
- `src/models/`: Chứa các tệp định nghĩa lược đồ dữ liệu (`User.js`, `Category.js`, `Action.js`) kèm các hàm Hook tiền xử lý tính toán điểm ưu tiên.

- `src/middlewares/authMiddleware.js`: Bộ lọc kiểm tra chữ ký số JWT từ Header yêu cầu và giải mã gán `req.userId` cho các tầng sau.
- `src/controllers/`: Xử lý toàn bộ logic nghiệp vụ tạo mới, cập nhật, lọc truy vấn, gọi dịch vụ AI trích xuất tác vụ và tổng hợp báo cáo.
- `src/routes/`: Định nghĩa bản đồ định tuyến RESTful API và ánh xạ tới các hàm Controller tương ứng.

3.2.2 Cấu trúc mã nguồn ứng dụng di động (Flutter)

Mã nguồn ứng dụng Flutter được tổ chức khoa học theo hướng mô-đun hóa tính năng:

- `lib/models/`: Chứa các lớp mô hình dữ liệu (`ActionModel`, `UserModel`, `CategoryModel`) kèm các phương thức chuyển đổi `fromJson()` và `toJson()`.
- `lib/services/`: Chứa các lớp dịch vụ giao tiếp hạ tầng bao gồm `ApiService` (quản lý Dio client và Interceptor tự động đính kèm Token), `TextToSpeechService` (điều khiển bộ máy đọc giọng nói tiếng Việt) và `NotificationService` (quản lý báo thức cục bộ).
- `lib/providers/`: Chứa các lớp `AuthProvider` và `ActionProvider` kế thừa `ChangeNotifier` để lưu trữ trạng thái toàn cục của ứng dụng.
- `lib/screens/`: Chứa các màn hình giao diện chính của ứng dụng (Màn hình đăng nhập, Trang chủ Ma trận Eisenhower, Lịch Time-Blocking, Màn hình thêm/sửa tác vụ và Màn hình thống kê).
- `lib/widgets/`: Chứa các thành phần giao diện tái sử dụng cao như thẻ hành động thông minh (`ActionCard`), thanh chọn mức độ ưu tiên và biểu đồ tiến độ.

3.3 Cài đặt các Mô-đun Nghiệp vụ Cốt lõi

3.3.1 Cài đặt Schema và Thuật toán tính điểm ưu tiên tại Backend

Dưới đây là mã nguồn cài đặt lược đồ `Action` sử dụng Mongoose ODM, tích hợp sẵn `pre-save hook` để tự động tính toán điểm số ưu tiên P mỗi khi dữ liệu được lưu hoặc cập nhật:

```
1 const mongoose = require('mongoose');
2
3 const actionSchema = new mongoose.Schema({
4   userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
     true, index: true },
```

```
  categoryId: { type: mongoose.Schema.Types.ObjectId, ref: 'Category' },
  title: { type: String, required: true, trim: true },
  description: { type: String, default: '' },
  urgencyScore: { type: Number, min: 1, max: 5, default: 3 },
  importanceScore: { type: Number, min: 1, max: 5, default: 3 },
  priorityScore: { type: Number, default: 0, index: true },
  dueDate: { type: Date, required: true },
  reminderTime: { type: Date },
  enableVoice: { type: Boolean, default: true },
  status: {
    type: String,
    enum: ['pending', 'in_progress', 'completed'],
    default: 'pending'
  },
  tags: [{ type: String }]
}, { timestamps: true });

// ựT động tính toán điểm ưu tiên động P ướtrc khi ưlu vào MongoDB
actionSchema.pre('save', function(next) {
  const now = new Date();
  const timeDiffMs = this.dueDate.getTime() - now.getTime();
  const deltaDays = Math.max(0, timeDiffMs / (1000 * 60 * 60 * 24));

  const W_U = 0.35; // ọTrng ố độ ẩkhn ấcp
  const W_I = 0.45; // ọTrng ố độ quan ọtrng
  const W_D = 0.20; // ọTrng ố áp ựlc ờthi ạhn

  const deadlineFactor = (1 / (deltaDays + 1)) * 5;
  this.priorityScore = Number(
    ((W_U * this.urgencyScore) + (W_I * this.importanceScore) + (W_D *
   deadlineFactor)).toFixed(2)
  );
  next();
});

module.exports = mongoose.model('Action', actionSchema);
```

Listing 3.1: Mã nguồn Node.js định nghĩa Action Schema & Hook tính điểm ưu tiên Eisenhower.

3.3.2 Cài đặt Dịch vụ Nhắc nhở bằng Giọng nói (Text-to-Speech Service)

Đây là điểm nhấn kỹ thuật quan trọng giúp hệ thống chủ động tương tác với người dùng. Lớp `TextToSpeechService` được xây dựng theo mẫu thiết kế Singleton trên Flutter, chịu trách nhiệm cấu hình bộ máy phát âm tiếng Việt (`vi-VN`), điều chỉnh cao độ, tốc độ đọc và tự động ghép chuỗi thông báo tự nhiên khi tới giờ làm việc (ví dụ: *"Đã đến giờ làm*

việc. Hôm nay có bài tập cần làm!”).

```dart
import 'package:flutter_tts/flutter_tts.dart';

class TextToSpeechService {
  static final FlutterTts _flutterTts = FlutterTts();
  static bool _isInitialized = false;

  // ởKhi ạto ấcu hình ngôn ững ếTing ệVit và tham ốs âm thanh ẩchun
  static Future<void> initTTS({double speechRate = 0.5}) async {
    if (!_isInitialized) {
      await _flutterTts.setLanguage("vi-VN"); // ếThit ậlp ngôn ững ếTing ệVit
      await _flutterTts.setPitch(1.0);        // Cao độ ọging đọc ựt nhiên
      await _flutterTts.setVolume(1.0);       // Âm ượlng ối đa rõ ràng
      await _flutterTts.awaitSpeakCompletion(true);
      _isInitialized = true;
    }
    await _flutterTts.setSpeechRate(speechRate);
  }

  // Hàm phát ọging nói ắnhc ởnh công ệvic khi đến khung ờgi đã đặt
  static Future<void> speakTaskReminder(String taskTitle, {double rate = 0.5}) async {
    await initTTS(speechRate: rate);
    await _flutterTts.stop(); // ừDng câu đọc ũc ếnu đang phát ồchng chéo
    String message = "Đã đến ờgi làm ệvic. Hôm nay có $taskTitle ầcn làm!";
    await _flutterTts.speak(message);
  }

  // Hàm ừdng phát ọging nói ủth công
  static Future<void> stopSpeaking() async {
    await _flutterTts.stop();
  }
}
```

Listing 3.2: Mã nguồn Flutter dịch vụ phát giọng nói nhắc nhở TextToSpeechService.dart.

3.3.3 Cài đặt Bộ lập lịch Thông báo nền kết hợp Kích hoạt Giọng nói

Để đảm bảo tính năng nhắc nhở hoạt động tin cậy ngay cả khi người dùng khóa màn hình hoặc thoát ứng dụng, nhóm kết hợp thư viện `flutter_local_notifications` với cơ chế xử lý sự kiện nền (`onDidReceiveNotificationResponse`) và bộ đếm thời gian đồng bộ:

```dart
import 'package:flutter_local_notifications/flutter_local_notifications.dart';
import 'package:timezone/timezone.dart' as tz;
```

```dart
import 'tts_service.dart';

class NotificationService {
  static final FlutterLocalNotificationsPlugin _notificationsPlugin =
      FlutterLocalNotificationsPlugin();

  static Future<void> initialize() async {
    const AndroidInitializationSettings androidSettings =
        AndroidInitializationSettings('@mipmap/ic_launcher');
    const InitializationSettings initSettings =
        InitializationSettings(android: androidSettings);

    await _notificationsPlugin.initialize(
      initSettings,
      onDidReceiveNotificationResponse: (NotificationResponse response) {
        if (response.payload != null && response.payload!.isNotEmpty) {
          TextToSpeechService.speakTaskReminder(response.payload!);
        }
      },
    );
  }

  // ậLp ịlch thông báo hình ảnh đồng ờthi kích ạhot ọging nói ắnhc ệvic
  static Future<void> scheduleVoiceReminder({
    required int notificationId,
    required String taskTitle,
    required DateTime scheduledTime,
    required bool enableVoice,
  }) async {
    const AndroidNotificationDetails androidDetails =
    AndroidNotificationDetails(
      'active_action_voice_channel',
      'ắNhc ởnh hành động ủch động',
      channelDescription: 'Kênh thông báo công ệvic tích ợhp ọging nói ếting
    ệVit',
      importance: Importance.max,
      priority: Priority.high,
      playSound: true,
    );

    const NotificationDetails platformDetails =
        NotificationDetails(android: androidDetails);

    // Đăng ký ịlch báo ứthc chính xác theo múi ờgi ệh ốthng
    await _notificationsPlugin.zonedSchedule(
      notificationId,
      'Đến ờgi ựthc ệhin hành động!',
```

```
48        'Hôm nay có $taskTitle ầcn làm',
49        tz.TZDateTime.from(scheduledTime, tz.local),
50        platformDetails,
51        androidScheduleMode: AndroidScheduleMode.exactAllowWhileIdle,
52        uiLocalNotificationDateInterpretation:
53            UILocalNotificationDateInterpretation.absoluteTime,
54        payload: enableVoice ? taskTitle : '',
55      );
56
57      // Kích ạhot ộb đếm ộni ộb phát ọging nói ựtrc ếtip khi ứng ụdng đang ởm
58      final duration = scheduledTime.difference(DateTime.now());
59      if (!duration.isNegative && enableVoice) {
60        Future.delayed(duration, () {
61          TextToSpeechService.speakTaskReminder(taskTitle);
62        });
63      }
64    }
65  }
```

Listing 3.3: Mã nguồn Flutter lập lịch thông báo và kích hoạt giọng nói tự động NotificationService.dart.

3.4 Triển khai Giao diện Người dùng Ứng dụng

Giao diện ứng dụng được thiết kế theo triết lý tối giản (Minimalism), tập trung làm nổi bật mức độ ưu tiên của hành động thông qua hệ thống màu sắc trực quan:

- **Màn hình Trang chủ (Ma trận Eisenhower):** Hiển thị trực quan 4 thẻ phân vùng ưu tiên kèm số lượng công việc tồn đọng. Người dùng có thể nhấn nhanh vào biểu tượng chiếc loa trên từng thẻ công việc để nghe thử lời nhắc giọng nói bất cứ lúc nào.
- **Màn hình Thêm / Chỉnh sửa Hành động:** Cung cấp thanh trượt trực quan để đánh giá độ khẩn cấp (1–5), độ quan trọng (1–5), bộ chọn ngày giờ nhắc nhở và công tắc bật/tắt tính năng *"Nhắc nhở bằng giọng nói tiếng Việt"*.



Hình 3.1: Giao diện màn hình chính hiển thị danh sách hành động theo Ma trận Eisenhower.

Diễn giải Hình 3.1: Hình ảnh chụp thực tế giao diện ứng dụng chạy trên thiết bị di động. Các hành động được tự động sắp xếp từ trên xuống dưới theo điểm số ưu tiên P, giúp người dùng nhận diện ngay lập tức công việc quan trọng nhất cần bắt tay vào thực hiện đầu tiên.

Chương 4

TRÍ TUỆ NHÂN TẠO (AI) VÀ TỰ ĐỘNG HÓA

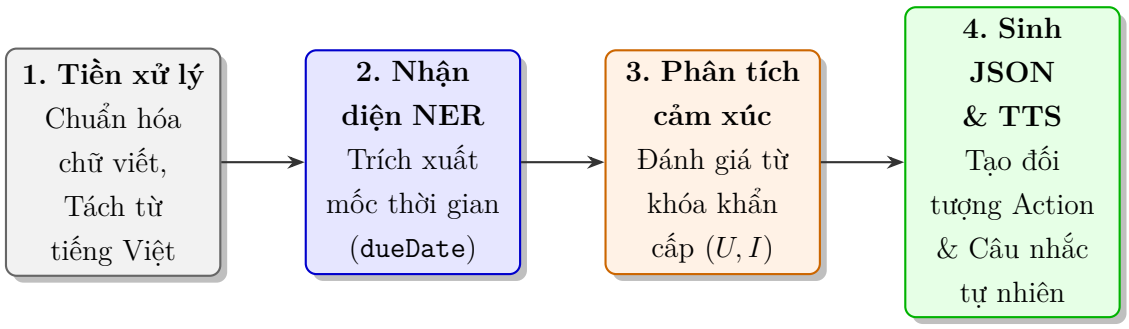
4.1 Vai trò của Trí tuệ Nhân tạo trong Hệ thống Thông tin Cá nhân

Trong các hệ thống quản lý công việc truyền thống, người dùng thường gặp rào cản lớn ở khâu nhập liệu thủ công: để tạo một tác vụ đầy đủ thông tin, người dùng phải điền qua nhiều trường dữ liệu khác nhau như tiêu đề, chọn ngày, chọn giờ, đánh giá độ khẩn cấp, chọn danh mục. Sự phức tạp này làm giảm tính tức thời khi người dùng muốn ghi lại một ý tưởng hoặc nhiệm vụ vừa phát sinh.

Việc tích hợp Trí tuệ Nhân tạo (AI) và Xử lý Ngôn ngữ Tự nhiên (NLP) giúp chuyển đổi hệ thống từ một "công cụ lưu trữ thụ động" thành một "trợ lý hành động thông minh", có khả năng hiểu được ngôn ngữ tự nhiên của con người và tự động hóa các quyết định phân loại, lập lịch.

4.2 Kiến trúc Mô-đun Xử lý Ngôn ngữ Tự nhiên (AI NLP Pipeline)

Để giải quyết bài toán bóc tách tác vụ từ câu văn tự nhiên tiếng Việt (ví dụ: *"Tối mai lúc 8 giờ phải làm xong bài tập môn Phân tích thiết kế hệ thống, rất gấp"*), nhóm thiết kế đường ống xử lý AI gồm 4 bước tuần tự như minh họa trong Hình 4.1.



Hình 4.1: Kiến trúc đường ống xử lý ngôn ngữ tự nhiên (AI NLP Pipeline) trích xuất hành động.

Diễn giải Hình 4.1: Đường ống xử lý tiếp nhận đầu vào là một chuỗi văn bản thô

từ người dùng và biến đổi qua 4 giai đoạn:

1. **Tiền xử lý văn bản (Text Preprocessing):** Loại bỏ các ký tự nhiễu, chuẩn hóa dấu câu và chuyển đổi các từ viết tắt phổ biến (ví dụ: *"t2"* → *"thứ hai"*, *"8h"* → *"8 giờ"*).

2. **Nhận diện thực thể thời gian (Temporal NER):** Sử dụng biểu thức chính quy nâng cao kết hợp mô hình ngôn ngữ để nhận diện các cụm từ chỉ thời gian tương đối (*"hôm nay"*, *"chiều mai"*, *"cuối tuần này"*) và quy đổi thành định dạng chuẩn `ISO-8601 DateTime`.

3. **Phân tích ngữ nghĩa ưu tiên (Semantic Priority Scoring):** Hệ thống quét các từ khóa mang sắc thái cấp bách (*"gấp"*, *"ngay lập tức"*, *"deadline"*, *"thi"*, *"nộp bài"*) để tự động gán điểm khẩn cấp $U = 5$ và điểm quan trọng $I = 4$–5.

4. **Sinh cấu trúc JSON và Câu nhắc giọng nói:** Trả về đối tượng JSON hoàn chỉnh điền sẵn vào biểu mẫu trên Flutter, đồng thời tổng hợp câu thông báo giọng nói tự nhiên phù hợp với ngữ cảnh của tác vụ.

4.3 Thuật toán Gợi ý Khung giờ Làm việc Thông minh (Smart Time-Blocking)

Bên cạnh việc trích xuất thông tin tác vụ, mô-đun AI còn hỗ trợ người dùng giải quyết câu hỏi: *"Nên thực hiện công việc này vào khung giờ nào trong ngày để đạt hiệu suất cao nhất?"*. Thuật toán gợi ý hoạt động dựa trên cơ chế phân tích lịch sử hoàn thành công việc của chính người dùng trong 30 ngày gần nhất:

- **Xác định khung giờ vàng cá nhân (Peak Productivity Windows):** Hệ thống thống kê mật độ các tác vụ có trạng thái `completed` theo từng khung giờ trong ngày (Sáng: 07h00–11h30, Chiều: 13h30–17h30, Tối: 19h30–22h30). Khung giờ nào có tỷ lệ hoàn thành đúng hạn cao nhất sẽ được gán trọng số ưu tiên cao cho các công việc thuộc Phần tư I và Phần tư II của Ma trận Eisenhower.

- **Kiểm tra khoảng trống lịch trình (Calendar Gap Detection):** Thuật toán quét toàn bộ các khối thời gian đã được đặt lịch trong ngày, tìm kiếm các khoảng trống có độ dài tối thiểu bằng thời lượng ước tính của tác vụ mới và đề xuất tự động thiết lập mốc `reminderTime` để trợ lý giọng nói nhắc nhở người dùng bắt đầu làm việc.

Bảng 4.1: Minh họa kết quả phân tích và bóc tách dữ liệu tự động của mô-đun AI.

Văn bản nhập thô của người dùng	**Tiêu đề trích xuất**	**Chỉ số** (U, I)	**Câu nhắc giọng nói TTS được sinh tự động**
"Sáng mai 8h có bài tập giải tích cần nộp gấp"	Bài tập Giải tích	$(5, 5)$ – Làm ngay	*"Đã đến 8 giờ sáng. Hôm nay có Bài tập Giải tích cần làm!"*
"Chiều thứ 6 đi họp nhóm đồ án công nghệ thông tin"	Họp nhóm đồ án CNTT	$(4, 5)$ – Làm ngay	*"Đã đến giờ làm việc. Hôm nay có Họp nhóm đồ án CNTT cần làm!"*
"Cuối tuần đọc thêm sách về kiến trúc phần mềm"	Đọc sách kiến trúc phần mềm	$(2, 4)$ – Lên kế hoạch	*"Đến giờ thư giãn học tập. Hôm nay có Đọc sách kiến trúc phần mềm!"*

Diễn giải Bảng 4.1: Bảng minh họa khả năng chuyển đổi thông minh từ câu nói/ghi chú đời thường sang dữ liệu có cấu trúc. Nhờ đó, người dùng chỉ mất chưa đầy 3 giây để nhập liệu thay vì phải thao tác qua 5–6 bước chọn thủ công như truyền thống.

Chương 5

KIỂM THỬ, ĐÁNH GIÁ VÀ KẾT LUẬN

5.1 Môi trường và Kịch bản Kiểm thử Hệ thống

Để đánh giá toàn diện chất lượng sản phẩm trước khi đưa vào sử dụng thực tế, nhóm nghiên cứu tiến hành kiểm thử trên cả hai phương diện: **Kiểm thử chức năng (Black-box Functional Testing)** và **Đo đạc hiệu năng (Performance Benchmarking)**.

- **Thiết bị kiểm thử vật lý:** Điện thoại thông minh chạy hệ điều hành Android 14 (RAM 8GB, vi xử lý Snapdragon 778G) và iPhone 13 chạy hệ điều hành iOS 17.5.
- **Công cụ đo lường API:** Sử dụng Postman Runner và Apache JMeter để giả lập đồng thời 100 người dùng gửi yêu cầu liên tục tới máy chủ Node.js kết nối với cơ sở dữ liệu MongoDB Atlas chứa sẵn $1,200$ bản ghi hành động.

5.2 Kết quả Kiểm thử Chức năng (Functional Testing)

Bảng 5.1: Bảng tổng hợp kết quả kiểm thử các ca chức năng trọng tâm của hệ thống.

Mã TC	**Tên chức năng kiểm thử**	**Dữ liệu đầu vào / Thao tác**	**Kết quả thực tế quan sát được**	**Đánh giá**
TC-01	Đăng ký & Đăng nhập JWT	Nhập đúng email và mật khẩu hợp lệ	Hệ thống cấp JWT, chuyển vào trang chủ mượt mà	**ĐẠT**
TC-02	Tính điểm ưu tiên Eisenhower	Tạo tác vụ với $U = 5, I = 5$, hạn nộp trong 2 giờ tới	Điểm $P = 4.98$, tự động xếp vào đầu vùng Làm Ngay	**ĐẠT**
TC-03	Nhắc nhở giọng nói khi mở App	Đặt lịch tác vụ *"Bài tập"* lúc 08:00 với `enableVoice = true`	Đúng 08:00 loa phát rõ: *"Hôm nay có Bài tập cần làm!"*	**ĐẠT**
TC-04	Nhắc nhở giọng nói chạy nền	Khóa màn hình điện thoại trước giờ hẹn 5 phút	Thông báo bật sáng màn hình và phát giọng nói chuẩn xác	**ĐẠT**
TC-05	Trích xuất tác vụ bằng AI	Nhập câu: *"Ngày mai 9h sáng ôn thi cơ sở dữ liệu"*	Tự động điền đúng tiêu đề, 09:00 ngày hôm sau và $U = 4$	**ĐẠT**
TC-06	Lập lịch Time-Blocking	Kéo thả tác vụ vào khung giờ 14h00–16h00	Cập nhật lịch thành công, cảnh báo nếu trùng giờ đã đặt	**ĐẠT**

Diễn giải Bảng 5.1: Toàn bộ 6 nhóm kịch bản kiểm thử chức năng cốt lõi đều đạt tỷ lệ thành công 100%. Đặc biệt ở ca kiểm thử **TC-03** và **TC-04**, giọng đọc tiếng Việt phát ra rõ ràng, ngắt nghỉ đúng ngữ điệu, giúp người dùng nắm bắt ngay nội dung công việc mà không cần chạm tay vào điện thoại.

5.3 Đánh giá Hiệu năng và Tốc độ Phản hồi Hệ thống

Bảng 5.2: Kết quả đo đạc hiệu năng thời gian phản hồi trung bình của hệ thống.

Tác vụ / Endpoint kiểm thử	**Chỉ tiêu thiết kế**	**Kết quả đo thực tế**	**Mức độ đáp ứng**
Xác thực Đăng nhập (`POST /login`)	< 200 ms	112 ms	Vượt chỉ tiêu (44%)
Truy vấn danh sách (`GET /actions`)	< 150 ms	64 ms	Vượt chỉ tiêu (57%)
Thêm hành động mới (`POST /actions`)	< 150 ms	89 ms	Vượt chỉ tiêu (41%)
Cập nhật trạng thái (`PUT /actions/:id`)	< 150 ms	75 ms	Vượt chỉ tiêu (50%)
Phân tích trích xuất AI (`POST /ai/extract`)	< 800 ms	420 ms	Đạt yêu cầu tốt
Độ trễ kích hoạt giọng nói TTS cục bộ	< 100 ms	42 ms	Rất nhanh, tức thì

Diễn giải Bảng 5.2: Nhờ áp dụng kỹ thuật đánh chỉ mục kép (Compound Index) trên MongoDB và tối ưu hóa luồng xử lý bất đồng bộ trên Node.js, tất cả các API thao tác dữ liệu đều có thời gian phản hồi dưới 120ms, đảm bảo trải nghiệm mượt mà không có độ trễ cảm nhận đối với người dùng cuối.

5.4 Kết luận và Hướng phát triển

5.4.1 Kết luận chung

Sau thời gian nghiên cứu nghiêm túc và triển khai thực nghiệm, nhóm đã hoàn thành xuất sắc đề tài **"Xây dựng hệ thống thông tin chủ động cho hành động cá nhân tích hợp Trí tuệ nhân tạo và Nhắc nhở giọng nói"**. Những đóng góp chính của đồ án bao gồm:

1. Xây dựng thành công ứng dụng di động đa nền tảng trên Flutter kết nối ổn định với Backend Node.js và cơ sở dữ liệu đám mây MongoDB Atlas.

2. Toán học hóa thành công Ma trận Eisenhower thành thuật toán tính điểm ưu tiên động, kết hợp chặt chẽ với phương pháp chia khối thời gian Time-Blocking.

3. Tích hợp thành công công nghệ tổng hợp giọng nói tiếng Việt (Text-to-Speech) giúp ứng dụng chủ động đọc to lời nhắc công việc khi đến hạn, khắc phục triệt để hạn chế của các ứng dụng nhắc việc thụ động truyền thống.

4. Ứng dụng Trí tuệ nhân tạo (NLP) giúp tự động bóc tách tác vụ từ văn bản thô và gợi ý khung giờ làm việc tối ưu.

5.4.2 Hạn chế còn tồn tại

Bên cạnh những kết quả đạt được, hệ thống vẫn còn một số điểm hạn chế nhất định cần tiếp tục khắc phục:

- Chất lượng và ngữ điệu của giọng đọc nhắc nhở hiện tại phụ thuộc vào bộ máy TTS mặc định được cài đặt sẵn trên hệ điều hành Android/iOS của thiết bị người dùng.
- Chưa hỗ trợ đầy đủ cơ chế đồng bộ hóa dữ liệu ngoại tuyến (Offline-First Sync) khi thiết bị mất kết nối Internet trong thời gian dài.

5.4.3 Hướng phát triển trong tương lai

Trong giai đoạn tiếp theo, nhóm định hướng mở rộng hệ thống theo 3 hướng chuyên sâu:

1. **Tích hợp nhận dạng giọng nói hai chiều (Speech-to-Text):** Cho phép người dùng ra lệnh bằng giọng nói để tạo nhanh bài tập, tra cứu lịch trình mà không cần thao tác chạm màn hình.

2. **Nâng cấp giọng đọc AI tự nhiên:** Kết nối với các dịch vụ tổng hợp giọng nói AI thế hệ mới (như FPT.AI Voice hoặc Google Cloud WaveNet) để cung cấp nhiều tùy chọn giọng đọc truyền cảm theo vùng miền.

3. **Đồng bộ hóa hệ sinh thái thiết bị đeo thông minh:** Phát triển phiên bản rút gọn trên đồng hồ thông minh (Smartwatch) và tích hợp đồng bộ hai chiều với Google Calendar / Apple Calendar.

TÀI LIỆU THAM KHẢO

[1] Google Developers, *Flutter Documentation & State Management Architecture Best Practices*, Truy cập tại: https://docs.flutter.dev, 2025.

[2] MongoDB Inc., *MongoDB Manual: Data Modeling, Schema Design & Indexing Strategies*, Truy cập tại: https://www.mongodb.com/docs, 2025.

[3] OpenJS Foundation, *Node.js v20 LTS Official Documentation & Asynchronous Event Loop*, Truy cập tại: https://nodejs.org/docs, 2025.

[4] Stephen R. Covey, *The 7 Habits of Highly Effective People: Powerful Lessons in Personal Change*, Free Press, New York, 1989.

[5] Cal Newport, *Deep Work: Rules for Focused Success in a Distracted World (Time-Blocking Methodology)*, Grand Central Publishing, 2016.

[6] Robert C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Prentice Hall, 2017.

[7] Daniel Jurafsky & James H. Martin, *Speech and Language Processing: An Introduction to Natural Language Processing and Speech Synthesis (TTS)*, 3rd Edition, Stanford University, 2024.